

Countdown with GRPO: post-training a 1.5B model, with a paired evaluation protocol

Qwen2.5-1.5B-Instruct, LoRA, 150 GRPO steps, single run

Summary

GRPO post-training of Qwen2.5-1.5B-Instruct (LoRA) on a Countdown arithmetic task, built around an evaluation protocol that supports claims of the form "model B beats model A" with a proper statistical test.

Headline: after 150 GRPO steps, greedy accuracy goes from 10.7% to 29.3% on 300 held-out puzzles (paired exact test, $p < 1e-7$). Most of the gain comes from the model producing valid expressions more often. Accuracy among valid expressions improves in greedy decoding but stays flat in sampling, and nothing significant shows up on harder puzzles. Details and limits below.

1. Task

Given three numbers in $[1, 10]$ and a target in $[2, 100]$, write an arithmetic expression using $+$ $-$ $*$ $/$ that evaluates to the target, using each given number at most once. The model answers in a fixed format: a reasoning part inside `<think>` tags, then the expression inside `<answer>` tags (for example `7 * 2 + 3`).

Puzzles are generated from a reachable target, so every puzzle is solvable (checked exhaustively on the evaluation set, see the null model). The prompt contains two solved examples and is applied through the chat template. Without the chat template the Instruct model never emits an end-of-sequence token and every completion is truncated.

2. Method

Item	Setting
Model	Qwen2.5-1.5B-Instruct, bf16
Adapter	LoRA, $r=16$, $\alpha=32$, dropout 0.05, all linear layers
Algorithm	GRPO (TRL), group size 8, 4 puzzles per step, learning rate $1e-5$, KL coefficient 0.04, temperature 1.0, 200 max completion tokens
Reward	Sum of three terms, weights 0.1 / 0.1 / 1.0: format (think and answer tags present), validity (only allowed characters and the puzzle's numbers, at most once each, at least two numbers, finite value), correctness (value equals the target)
Length	150 steps in total, about 600 puzzles seen

The three-term reward exists because of what a first iteration showed: with an all-or-nothing reward and a 0/1 format check, close to half of the outputs failed before the value was even compared (forbidden characters such as `"= 50"` inside the answer tag, invented numbers), and nothing distinguished them from a valid but wrong expression.

3. Evaluation protocol

- **Fixed, deduplicated evaluation sets.** 300 standard puzzles (fixed seed, fingerprint aab1c2acbf), excluded from the training puzzles and distinct from each other. The puzzle space is small, about 2,500 distinct 3-number puzzles, so leakage between train and eval is a real risk and is checked. Two results are only compared if their fingerprints match.
- **Hard set.** 100 puzzles with 4 numbers in $[1, 25]$ and targets up to 999 (fingerprint 98a8a4ebb5), to test transfer beyond the training distribution.

- **Greedy decoding** on 300 puzzles, and **sampling** (4 completions per puzzle) on the first 200, at temperature 1.0 (the training temperature). Temperature 0.7 was also run once, for the baseline and the 60-step checkpoint.
- **Paired statistics.** Both models are evaluated on the same puzzles. Greedy: exact McNemar test and a bootstrap interval on the difference. Sampling: bootstrap over puzzles (the 4 completions of a puzzle are correlated) and an exact sign test on per-puzzle mean accuracy.
- **Failure stages**, for each output: broken format, forbidden characters, wrong numbers, wrong value, correct. "Valid" below means the output reaches the "wrong value" or "correct" stage.
- Per-puzzle results, including completions for greedy runs, are saved to JSON so any comparison can be recomputed.

4. Results

4.1 Standard puzzles (3 numbers)

Measure	Baseline	60 steps	150 steps	Baseline vs 150 steps
Greedy, 300 puzzles	10.7% (32/300)	10.0% (30/300)	29.3% (88/300)	+18.7 pts, 95% CI [+12.7, +24.7], McNemar 75 vs 19, $p < 1e-7$
Sampled T=1.0, 200 x 4	10.1%	15.1%	20.8%	+10.6 pts, CI [+7.2, +14.0], sign test 75 vs 22, $p < 1e-6$
pass@4, T=1.0	32.0%	43.5%	50.5%	not tested

Intermediate comparisons. Baseline vs 60 steps, sampled T=1.0: +5.0 pts, CI [+2.0, +8.1], $p = 0.0095$ (T=0.7: +4.6 pts, $p = 0.016$). 60 vs 150 steps, sampled T=1.0: +5.6 pts, CI [+2.2, +9.1], $p = 0.012$. Greedy, baseline vs 60 steps: no difference ($p = 0.86$).

Failure stages, greedy, counts out of 300 (broken format / forbidden characters / wrong numbers / wrong value / correct):

	Baseline	150 steps
Count	7 / 64 / 75 / 122 / 32	2 / 1 / 15 / 194 / 88

4.2 Where the gain comes from

Accuracy factors as $P(\text{valid}) \times P(\text{correct} \mid \text{valid})$.

Setting	Model	P(valid)	P(correct valid)	Accuracy
Sampled T=1.0	Baseline	36.4% (291/800)	27.8% (81/291)	10.1%
	60 steps	49.0% (392/800)	30.9% (121/392)	15.1%
	150 steps	76.6% (613/800)	27.1% (166/613)	20.8%
Greedy	Baseline	51.3% (154/300)	20.8% (32/154)	10.7%
	60 steps	60.7%	16.5%	10.0%
	150 steps	94.0% (282/300)	31.2% (88/282)	29.3%

- **Sampling:** validity more than doubles while the conditional accuracy stays between 27% and 31%. At 150 steps, about +11.2 points of the +10.6 gain are explained by validity and about -0.5 by the conditional accuracy.
- **Greedy:** the conditional accuracy rises from 20.8% (95% CI [15.1, 27.9]) to 31.2% ([26.1, 36.8]). The difference is detectable but its size is uncertain (unpaired test p of about 0.02, difference CI [+1.7, +18.4]). On the 147 puzzles that are valid under both models, the baseline solves 29 and the 150-step model 53 (40 gained vs 16 lost, exact $p = 0.002$). This subset is defined after the fact, so it is exploratory.

- A possible reading, not tested here: the baseline's greedy path was worse than its average sample (20.8% vs 27.8% when valid), and training moved the greedy path to roughly the level of an average sample.

4.3 Null model

An exact enumeration of every expression for each evaluation puzzle gives the accuracy of a uniformly random valid expression: 3.8% when all numbers are used, 2.7% when the number of operands is free (`null_model.py`). The model's accuracy given validity is 27% to 31%, about 7 to 10 times higher, so the baseline already has some arithmetic intuition. It has not clearly increased in sampling.

4.4 Hard puzzles (4 numbers, 100 puzzles)

Measure	Baseline	150 steps	Difference
Greedy	3.0%	7.0%	+4.0 pts, CI [-1.0, +10.0], McNemar 6 vs 2, $p = 0.29$
Sampled $T=1.0$, 100×4	3.2%	5.8%	+2.5 pts, CI [-0.2, +5.2], sign test 16 vs 8, $p = 0.15$
pass@4	13.0%	20.0%	not tested

Not significant, and both models solve very few puzzles, so these tests have little power. The failure stages show the validity gain carrying over (at least 39 of 100 greedy outputs move from an early failure to a valid but wrong expression), with no demonstrated gain in correctness.

4.5 Training signal

During the first 60 steps the logged mean correctness reward rises from 0.147 (steps 1 to 10) to 0.231 (steps 51 to 60); a regression over the 60 steps gives a slope of +0.002 per step ($p = 0.004$). The ratio of correctness to validity reward does not rise significantly between steps 1 to 20 and steps 41 to 60 (0.335 to 0.403, bootstrap CI of the difference [-0.03, +0.17]).

5. Limitations

- One training run, one seed. No estimate of run-to-run variance.
- The final configuration changed several things at once relative to the first iteration (three-term reward, 4 puzzles per step instead of 1, no repetition penalty), so the improvement cannot be attributed to the reward shaping alone. That first iteration, with a two-term reward, a repetition penalty and one puzzle per update, showed no gain over the baseline.
- Hardware and library versions differ inside the run. Steps 1 to 100 ran with a TRL version that was not recorded (PEFT 0.21.1). Steps 101 to 150 ran on a T4 with TRL 1.15.0, where a bf16-to-fp32 matrix multiplication path in TRL's chunked log-probability kernel is unsupported, and `_MM_OUT_DTYPE` was set to False to fall back to a float32 computation. This path was not otherwise validated.
- Small puzzle space (about 2,500 distinct puzzles with 3 numbers). Train and evaluation puzzles are distinct, but the evaluation distribution is the training distribution.
- The hard-set results are underpowered.
- The explanations of the mechanism (validity first, then sharper greedy path) are hypotheses. The ablation that would test them, training without the validity term, was not run.
- The "valid" stage used in the analysis is slightly broader than the validity term of the reward (for example, a one-number answer reaches the "wrong value" stage).
- The evaluation prompt is the training prompt. Robustness to prompt changes is not measured.

6. Reproduction

Install: `pip install trl transformers datasets accelerate peft`. Training is resumable from Drive checkpoints; `max_steps` is read from the new configuration, not from the checkpoint.

```

from trl import GRPOTrainer
from train_countdown_grpo import (build_hf_dataset, make_config,
    REWARD_FUNCS, peft_config, MODEL_NAME)

OUT = "/path/to/output"
config = make_config(output_dir=OUT, max_steps=60) # first phase
trainer = GRPOTrainer(model=MODEL_NAME, reward_funcs=REWARD_FUNCS, args=config,
    train_dataset=build_hf_dataset(), peft_config=peft_config)
trainer.train()

# continue to 150 steps
config = make_config(output_dir=OUT, max_steps=150)
trainer = GRPOTrainer(model=MODEL_NAME, reward_funcs=REWARD_FUNCS, args=config,
    train_dataset=build_hf_dataset(), peft_config=peft_config)
trainer.train(resume_from_checkpoint=f"{OUT}/checkpoint-60")

```

Evaluation and comparisons:

```

# greedy, 300 puzzles
python eval_countdown.py --mode mid --tag baseline-mid --out-dir results
python eval_countdown.py --mode mid --adapter $OUT/checkpoint-150 --tag step150-mid --out-dir results
# sampling, T=1.0, 200 puzzles x 4
python eval_countdown.py --mode samp --temperature 1.0 --seed 0 --skip-greedy --tag baseline-T1 --out-dir results
python eval_countdown.py --mode samp --temperature 1.0 --seed 0 --skip-greedy \
    --adapter $OUT/checkpoint-150 --tag step150-T1 --out-dir results
# hard puzzles
python eval_countdown.py --mode hard --temperature 1.0 --seed 0 --tag baseline-hard --out-dir results
python eval_countdown.py --mode hard --temperature 1.0 --seed 0 \
    --adapter $OUT/checkpoint-150 --tag step150-hard --out-dir results
# paired comparisons (baseline first), sections: greedy, sampled, hard, hard_sampled
python compare_evals.py BASELINE.json STEP150.json --section greedy
# null model
python null_model.py

```

Files: `countdown_grpo_starter.py` (puzzles, prompt, rewards), `train_countdown_grpo.py` (configuration, training), `eval_countdown.py` (standardised evaluation), `compare_evals.py` (paired comparison), `null_model.py` (random valid expression baseline). Record library versions next to the results.

7. Next steps

- Cold-start supervised fine-tuning on search traces produced by an exhaustive solver, followed by GRPO, evaluated with the same protocol. The target metric is accuracy given validity, which has not moved reliably in sampling.
- Ablation without the validity term, and a larger group size, to test whether the correctness signal is being crowded out.
- Several seeds, and more hard puzzles for a test with usable power.